

■ LAST GENERATED RANDOM NUMBER

0x18BDC6BC

DECIMAL
415090364

GENERATED AT
22/04/2026, 14:27:45

BIT LENGTH
32 bits

0001000 1011101 11000110 1011100

■ ENTROPY HEALTH



QUANTUM OK

BIAS SCORE	TOTAL BITS
0.0156	64
ONES	ZEROS
33	31

■ SELECTED RANGE

FROM

15 / 04 / 2026 

TO

22 / 04 / 2026 

APPLY

4

PACKETS

1.56%

BIAS |P(1)-50%|

► DEVICE

TYPE ESP32-S3

SENSOR RadonEye RD200V2

PROTOCOL BLE 4.2

SOURCE RadonEye RD200V2 • BLE C-Pulse

SERVER TIME 2026-04-22 20:59:21 UTC

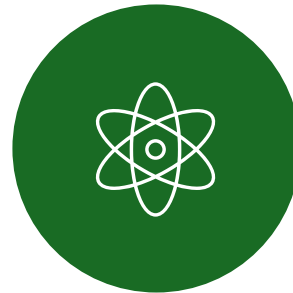
SERVICE UUID

00001523-0000-1000-8000-00005F9b34Fb

True Randomness From Quantum Physics



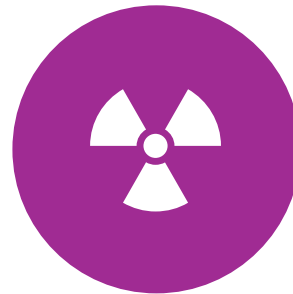
TRNGs use classical physical processes (e.g., thermal noise, shot noise) which can be affected by the environment.



A QRNG is different. It uses a physical quantum event as the entropy source.

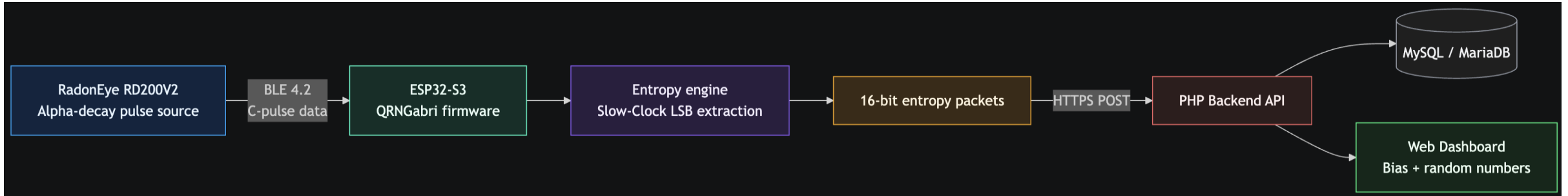


In QRNGabri, that source is Radon-222 alpha decay.



The exact instant a radioactive atom decays is fundamentally unpredictable, which makes it a genuine quantum source of randomness.

My QRNGabri Setup



I used a RadonEye RD200V2 as the radiation detector and an ESP32-S3 as the acquisition device.



The ESP32 connects to the RadonEye over BLE and reads the detector's C-pulse counters.



Those measurements are converted into entropy bits and sent to a backend over HTTPS.



The backend stores the data, computes health and bias metrics, and publishes live random numbers on a dashboard.

From Decay Events To Random Numbers

